Software Test Plan (STP)

Snake Game

Project: Snake Game (Console and Graphics Interface)
Version: 1.0
Author: Pradyumna
SRN: PES2UG24CS359
Course: Software Engineering – Mini Project
Date: 16-09-2026

Project: Snake Game (Console and Graphics Interface)
Version: 1.0
Author: Noel
SRN: PES2UG24CS330
Course: Software Engineering – Mini Project
Date: 16-09-2026

Project: Snake Game (Console and Graphics Interface)
Version: 1.0
Author: Sudeshna
SRN: PES2UG24CS313
Course: Software Engineering – Mini Project
Date: 16-09-2026

Project: Snake Game (Console and Graphics Interface)
Version: 1.0
Author: Shyam
SRN: PES2UG24CS349
Course: Software Engineering – Mini Project
Date: 16-09-2026

1. Introduction

Purpose: This document defines the test plan for the Snake Game mini project v1.0. It outlines objectives, scope, strategy, resources, schedule, and responsibilities for testing.

Scope: Testing covers Snake Game features such as interface mode selection, snake movement and control, food generation and growth, obstacle/wall/self-collision detection, score tracking, and session control (pause, resume, restart, exit), across both console and graphics interface modes. Compiler internals, OS-level keyboard drivers, and third-party graphics/console library internals are excluded.

References: Snake Game SRS v1.0, Snake Game Architecture (Component Diagram & Justification) v1.0.

Definitions: FR (Functional Requirement), NFR (Non-Functional Requirement), SRS (Software Requirements Specification), RTM (Requirements Traceability Matrix), CLI (Console/Command-Line Interface), GUI (Graphical User Interface).

2. Test Items

- Interface mode selection module (console / graphics)
- Snake movement and control module
- Food generation and growth module
- Obstacle and boundary collision module
- Self-collision and score tracking module
- Game-over, restart, and session control module

3. Features to be Tested

Features mapped to SRS requirement IDs:

- SNK-F-001: Interface mode selection menu
- SNK-F-004 / SNK-F-006: Snake movement and direction control
- SNK-F-009: Food consumption and snake growth
- SNK-F-012 / SNK-F-013: Wall and obstacle collision → game over
- SNK-F-014: Self-collision → game over
- SNK-F-015 / SNK-F-016: Score increment and display
- SNK-F-018: Restart session
- SNK-NF-001: Tick processing time ≤ 50 ms
- SNK-NF-002: No crash/hang during a 30-minute session
- SNK-SR-001 / SNK-SR-002: Bounds-checked indices and safe input handling

4. Features Not to be Tested

- C/C++ compiler internals and standard library correctness (assumed correct)
- Operating system keyboard driver behavior (vendor/OS responsibility)
- Internal rendering pipeline of the console or graphics library used (e.g., ncurses/SDL2 internals)

5. Test Approach / Strategy

Levels:

- Unit tests (module-level: movement, collision, scoring logic)
- Integration tests (Game Engine ↔ Collision Detection ↔ Score/Session Manager ↔ Game State Store)
- System tests (end-to-end gameplay in both console and graphics modes)
- Acceptance tests (informal walkthrough by team/instructor)

Types:

- Functional testing (core gameplay features)
- Regression testing (after each feature addition/bug fix)
- Performance testing (tick latency, frame timing)
- Usability testing (control responsiveness, on-screen clarity in both modes)

Entry Criteria: Stable build compiles cleanly on target OS, test data (obstacle layout, keyboard mapping) available, test environment ready.

Exit Criteria: 100% of planned test cases executed, 0 critical defects open, all High-priority acceptance criteria satisfied.

5.1 Security Validation

- Validate array-bounds checking for snake body, food, and obstacle positions
- Verify no unsafe/unbounded input functions (e.g., gets()) are used
- Fuzz keyboard input (rapid, invalid, and simultaneous key presses) to confirm no crash
- Static analysis pass (e.g., cppcheck) to confirm no unchecked pointer arithmetic
- Verify no open network sockets during execution

6. Test Environment

Hardware: Standard desktop/laptop with a 101-key keyboard and display monitor.

Software: Snake Game build (C/C++), console library (conio.h/ncurses), graphics library (graphics.h/SDL2), GCC/G++ or MSVC compiler, Windows and/or Linux OS.

Tools: Manual test execution, Valgrind or AddressSanitizer (memory-safety checks), cppcheck (static analysis), Git (version control), GitHub Issues (defect tracking).

Test Data: A fixed obstacle layout, a defined starting snake position, and a set of scripted keyboard input sequences.

7. Test Schedule

Milestones:

- Test case design: TBD
- Environment setup: TBD
- Test execution start: TBD
- Test execution end: TBD
- Final review / walkthrough: TBD

8. Test Deliverables

- Test Plan (this document)
- Test Cases (manual)
- Test Data
- Test Execution Logs
- Defect Reports
- Test Summary Report

9. Roles and Responsibilities

Role	Name	Responsibility
QA Lead	Pradyumna	Prepare plan, coordinate execution
Test Engineer	Noel	Design & execute test cases, log defects
Developer	Sudeshna	Support defect fixes and triage
Product Owner	Shyam	Approve test results, sign-off readiness

10. Risks and Mitigation

Risk	Mitigation
Delay in stable build delivery	Request early builds from whoever owns the affected module
Inconsistent behaviour across Windows/Linux or console/graphics modes	Test both interface modes on both target OS platforms before sign-off
Limited time before submission deadline	Prioritize High-priority FRs and NFRs first; defer Medium-priority edge cases if needed

11. Assumptions & Dependencies

- The build compiles without errors on at least one target OS before test execution begins
- All 4 team members have access to a working development/test environment
- The SRS and Component Diagram remain stable during the testing window (no late requirement changes)

12. Suspension & Resumption Criteria

Suspend testing if:

- The build fails to compile or crashes on startup, blocking more than 30% of planned test cases

Resume testing if:

- The blocking defect is fixed and a new stable build is provided

13. Test Case Management & Traceability

The RTM in the SRS document ensures mapping of requirements to test cases. The table below lists the test cases designed for this Test Plan, covering both functional and non-functional requirements.

TC ID	Req ID	Description	Test Steps	Expected Result
TC-Menu-01	SNK-F-001	Interface mode menu appears and accepts selection	1. Launch application. 2. Observe startup menu. 3. Select console or graphics mode.	Menu is displayed before gameplay; chosen mode loads correctly.
TC-Move-01	SNK-F-004	Snake moves one cell per tick	1. Start game. 2. Observe snake position over several ticks without pressing any key.	Snake advances exactly one grid cell per tick in its current direction.
TC-Move-03	SNK-F-006	Snake cannot reverse into itself	1. Start game moving Right. 2. Press the Left (opposite) key immediately.	Direction is unchanged; snake continues moving Right, no immediate self-collision.
TC-Food-02	SNK-F-009	Snake grows after eating food	1. Steer snake head onto the food cell. 2. Observe snake length before and after.	Snake length increases by exactly one segment.
TC-Obs-02	SNK-F-012	Game over on wall collision	1. Steer snake directly into the boundary wall.	Game-over screen is triggered immediately on wall contact.
TC-Obs-03	SNK-F-013	Game over on obstacle collision	1. Steer snake directly into a static obstacle cell.	Game-over screen is triggered immediately on obstacle contact.
TC-Score-01	SNK-F-014	Game over on self-collision	1. Grow the snake to at least 4 segments. 2. Steer the head into the snake's own body.	Game-over screen is triggered immediately on self-collision.
TC-Session-02	SNK-F-018	Restart resets game state	1. Play until game over. 2. Select Restart from the game-over screen.	Snake, score, food, and obstacles reset to their initial state; new session begins.
TC-Perf-01	SNK-NF-001	Tick processing time within budget	1. Instrument the game loop. 2. Run a 2-minute play session. 3. Record tick processing time.	At least 95% of ticks complete in ≤ 50 ms.
TC-Sec-01	SNK-SR-001 / SNK-SR-002	No crash on rapid/invalid keyboard input	1. Rapidly send a mix of valid, invalid, and simultaneous key presses for 1 minute.	Application does not crash or corrupt game state; invalid input is ignored.

14. Test Metrics & Reporting

- % test cases executed
- % passed / failed
- Defect count and severity
- Requirement coverage (FRs and NFRs mapped to at least one test case)

Reports: End-of-session execution status; final Test Summary Report before submission.

15. Approvals

Role	Name	Signature / Date
QA Lead	Pradyumna	
Dev Lead	Sudeshna	
Product Owner	Shyam	